

Axel BLOIS
Afouane MOUHAMAD

Geometry Modelling, Spring, 2025

TD 2 – Mesh Subdivision

Geometry Modelling, Spring, 2025	1
TD 2 – Mesh Subdivision	1
Question 1 :	2
Question 2 :	3
Question 3 :	4

Question 1 :

Après une itération de Catmull-Clark, un maillage avec n sommets, m arêtes et f faces aura :

- $n+m+f$ sommets (anciens sommets + 1 par arête + 1 par face),
- un nombre d'arêtes et de faces dépendant du nombre de côtés par face.

Si le maillage est entièrement en quadrilatères, alors :

- $2m$ arêtes (chaque arête est divisée),
- $4f$ faces (chaque face devient 4 petites faces).

Ce procédé rend le maillage plus lisse et plus détaillé.

after one iteration of the Catmull-Clark subdivision, a mesh with n vertices, m edges, and f faces will have:

- $n + m + f$ vertices (original vertices + one new vertex per edge + one new vertex per face),

- a number of edges and faces that depends on the number of sides per face.

If the mesh is made entirely of quadrilaterals, then:

- $2m$ edges (each edge is split into two),
- $4f$ faces (each face is subdivided into 4 smaller faces).

This process makes the mesh smoother and more detailed.

Question 2 :

```
1  v fonction splitHalfedge(e):
2      // on récupère les deux extrémités de l'arête
3      A = e.origin
4      B = e.twin.origin
5      V = (A + B) / 2 // nouveau sommet au milieu de l'arête
6
7      Créer 4 nouvelles demi-arêtes : e1 (A→V), e2 (V→A), e3 (V→B), e4 (B→V)
8      Créer 2 nouvelles arêtes entre (A,V) et (V,B)
9
10     Relier correctement les demi-arêtes avec next, twin, face, etc.
11     Supprimer e et e.twin
12     Ajouter les nouvelles arêtes et demi-arêtes dans le maillage
13
14
```

Question 3 :

```

1  fonction splitFace(f, M):
2      // 1. Récupérer les sommets de la face f dans l'ordre
3      sommets = []
4      he = f.halfedge
5      faire
6          sommets.ajouter(he.origin)
7          he = he.next
8      tant que he != f.halfedge
9
10     // 2. Calculer le centroïde c de la face
11     c = nouveau Sommet
12     c.position = (0, 0, 0)
13     pour chaque sommet s dans sommets:
14         c.position += s.position
15     c.position = c.position / taille(sommets)
16     M.ajouterSommet(c)
17
18     // 3. Pour chaque paire de sommets consécutifs, créer un triangle avec
19     pour i de 0 à taille(sommets)-1:
20         A = sommets[i]
21         B = sommets[(i+1) % taille(sommets)]
22
23         // Créer une nouvelle face triangulaire avec A, B, et c
24         new_face = nouvelle Face
25
26         // Créer les 3 demi-arêtes du triangle
27         he1 = nouvelle DemiArete (origine c)
28         he2 = nouvelle DemiArete (origine A)
29         he3 = nouvelle DemiArete (origine B)
30
31         // Relier les demi-arêtes entre elles
32         he1.next = he2
33         he2.next = he3
34         he3.next = he1
35
36         // Assigner la face à chaque demi-arête
37         he1.face = new_face
38         he2.face = new_face
39         he3.face = new_face
40
41         // Ajouter les demi-arêtes et la face au maillage
42         M.ajouterDemiArete(he1)
43         M.ajouterDemiArete(he2)
44         M.ajouterDemiArete(he3)
45         M.ajouterFace(new_face)
46
47     // 4. Supprimer la face d'origine f
48     M.supprimerFace(f)
49

```

Cette fonction divise une face polygonale en triangles autour de son centroïde. Elle calcule d'abord le centroïde, puis crée un nouveau sommet à cette position. Ensuite, elle crée autant de triangles que de côtés de la face, chacun formé par

le nouveau sommet et deux sommets consécutifs de la face.

La face initiale est supprimée, remplacée par ces triangles, ce qui permet de simplifier la géométrie et facilite les opérations ultérieures sur le maillage.

Question 4 :

```
fonction collapseEdge(M, e):
    A = e.origin           // sommet A au début de e
    B = e.twin.origin       // sommet B à l'autre bout de e

    // calculer la position du nouveau sommet (milieu de A et B)
    newPos = (A.position + B.position) / 2
    A.position = newPos    // mettre à jour A avec la nouvelle position

    // reconnecter toutes les demi-arêtes qui partaient de B vers A
    pour chaque demi-arête he partant de B:
        he.origin = A

    M.supprimerSommet(B)    // supprimer le sommet B

    // supprimer e et sa jumelle
    M.supprimerDemiArete(e)
    M.supprimerDemiArete(e.twin)

    // supprimer les arêtes en double (qui vont 2 fois vers A)
    pour chaque arête dans M:
        si c'est une arête en double ou une boucle sur A:
            M.supprimerArete(cette arête)

    // fin de la fonction, la topologie est mise à jour
```

Cette fonction effondre une demi-arête en fusionnant ses deux sommets en un seul. La position du nouveau sommet est calculée comme le milieu des deux anciens sommets. Toutes les arêtes qui étaient connectées au sommet supprimé sont reconnectées au nouveau sommet. La demi-arête effondrée ainsi que sa jumelle sont supprimées. Ensuite, on supprime les arêtes en double qui peuvent apparaître à cause de la fusion. Cette opération simplifie le maillage tout en évitant les erreurs topologiques.

Question 5 :